match starts. However, the sequences \1, \2, and so on, may be used to interpolate successive numbered groups in the match, and \k<_name_> will substitute the corresponding named captures. There are a couple of other regular expression bits that can be included in the replacement string, see the full Ruby documentation for details.

If the replacement is a block, the block is called with the matched part of the string and the value of the block is the replacement:

```
"The pickaxe book".gsub(/[aeiou]/) { |str| str.upcase }  # => "ThE pIckAxE bOOk"
```

If the replacement is a Hash, the keys should be strings, and each matched text is replaced by the value associated with that key in the hash, or just removed if there is no matching key – note that the keys have to be strings, so we need hashrocket syntax here.

```
"The pickaxe book".gsub(/[aeiou]/, {"a" => "b", "e" => "f"})  # => "Thf pckbxf
                                                              # .. bk"
```

The gsub! method has the same behavior as gsub except that it mutates its receiver.

The sub method is like gsub except that it only replaces the first match in the string— the "g" in gsub is for "global." The sub! method is like sub except that it mutates its receiver.

When you want to replace a specific character with a specific other character, Ruby provides the tr method, which should be faster for the cases in which it applies. The tr method is probably most commonly used with two one character strings as arguments, it converts all instances of the first string into the second.

But the method is actually more powerful: the first argument can be any character selector (as in Finding Information about a String, on page 518) and the second argument is a string of replacements. It returns a copy of the receiving string with the characters in the first argument replaced by the corresponding characters in the second argument. If the second argument is shorter than the first argument, it is padded with its last character. Both strings may use all the character selector syntax. For example:

```
"hello".tr("l", "$")      # => "he$$o"
"hello".tr("aeiou", "*")  # => "h*ll*"
"hello".tr("^aeiou", "*") # => "*e**o"
"hello".tr("el", "ip")    # => "hippo"
"hello".tr("a-y", "b-z")  # => "ifmmp"
```

The related tr! method has the same behavior but changes the receiving string in place.

The squeeze method takes a list of character selectors (as in Finding Information about a String, on page 518) and replaces any case where more than one of the characters happens in a row with a single instance. It's most commonly used as squeeze(" ") to clean up extra spaces. The squeeze! method changes the receiver in place.

Formatting Strings

A trio of methods pad a string inside a larger size: ljust, which left justifies the string; rjust, which right justifies it; and center, which places the receiving string inside a larger size. All three take two arguments: the size of the larger string and an optional string to pad the space. A space character is the default.

```
"book".center(15)      # => "     book      "
"book".ljust(15)       # => "book           "
```